HW2

65090502417 Piyapat Jantawee

2025-02-21

ข้อ1

Data : BankChurners

Class Variable : Attrition Flag

```
bank <- read.csv('BankChurners.csv')
table(bank$Attrition_Flag)

##
## Attrited Customer Existing Customer
##              1627              8500

str(factor(bank$Attrition_Flag))

##  Factor w/ 2 levels "Attrited Customer",..: 2 2 2 2 2 2 2 2 2 2 ...
```

“ยัด data ทั้งหมด(ให้ y = Attrition Flag (ปรับเป็น factor variable โดยที่ Existing = 1 Attrited = 0) x = ทุกตัวแปรที่เหลือยกเว้น CLIENTNUM) เข้า model ดูเทียบค่า P-value ตรวจสอบว่าว่าค่าใดควรอยู่ในตัวแบบ”

```
library(base)

bank <- bank[2:length(bank)]

number <- dim(bank)[1]
bank$Attrition_Flag <- factor(bank$Attrition_Flag, levels = c("Attrited
Customer","Existing Customer"))

summary(glm(Attrition_Flag~. ,data = bank, family = 'binomial'))

##
## Call:
## glm(formula = Attrition_Flag ~ ., family = "binomial", data = bank)
##
## Coefficients: (1 not defined because of singularities)
##                                Estimate Std. Error z value Pr(>|z|)
## (Intercept)                  -6.723e+00  4.748e-01 -14.162  < 2e-16 ***
## Customer_Age                  6.131e-03  7.711e-03   0.795 0.426532
## GenderM                       8.938e-01  1.455e-01   6.142 8.16e-10 ***
## Dependent_count              -1.358e-01  2.998e-02  -4.530 5.89e-06 ***
## Education_LevelDoctorate     -3.689e-01  2.081e-01  -1.773 0.076218 .
```

```
## Education_LevelGraduate         5.798e-03  1.396e-01   0.042 0.966864
## Education_LevelHigh School     -1.026e-02  1.488e-01  -0.069 0.945016
## Education_LevelPost-Graduate   -3.112e-01  2.050e-01  -1.518 0.128952
## Education_LevelUneducated      -6.955e-02  1.573e-01  -0.442 0.658477
## Education_LevelUnknown         -1.329e-01  1.554e-01  -0.855 0.392310
## Marital_StatusMarried           4.994e-01  1.544e-01   3.234 0.001219 **
## Marital_StatusSingle           -1.081e-01  1.549e-01  -0.698 0.485248
## Marital_StatusUnknown          -4.528e-02  1.962e-01  -0.231 0.817467
## Income_Category$40K - $60K      9.083e-01  2.026e-01   4.484 7.33e-06 ***
## Income_Category$60K - $80K      6.405e-01  1.791e-01   3.576 0.000349 ***
## Income_Category$80K - $120K     2.983e-01  1.663e-01   1.794 0.072811 .
## Income_CategoryLess than $40K   7.702e-01  2.190e-01   3.516 0.000438 ***
## Income_CategoryUnknown          8.321e-01  2.322e-01   3.584 0.000338 ***
## Card_CategoryGold              -1.066e+00  3.521e-01  -3.026 0.002475 **
## Card_CategoryPlatinum          -9.816e-01  6.813e-01  -1.441 0.149654
## Card_CategorySilver            -4.502e-01  1.962e-01  -2.294 0.021778 *
## Months_on_book                  4.685e-03  7.673e-03   0.611 0.541484
## Total_Relationship_Count        4.493e-01  2.750e-02  16.338  < 2e-16 ***
## Months_Inactive_12_mon         -5.078e-01  3.793e-02 -13.387  < 2e-16 ***
## Contacts_Count_12_mon          -5.133e-01  3.655e-02 -14.044  < 2e-16 ***
## Credit_Limit                    1.971e-05  6.860e-06   2.873 0.004064 **
## Total_Revolving_Bal             9.321e-04  7.207e-05  12.934  < 2e-16 ***
## Avg_Open_To_Buy                        NA         NA      NA       NA
## Total_Amt_Chng_Q4_Q1            4.262e-01  1.878e-01   2.269 0.023253 *
## Total_Trans_Amt                -4.855e-04  2.295e-05 -21.154  < 2e-16 ***
## Total_Trans_Ct                  1.192e-01  3.731e-03  31.944  < 2e-16 ***
## Total_Ct_Chng_Q4_Q1             2.798e+00  1.889e-01  14.813  < 2e-16 ***
## Avg_Utilization_Ratio           1.253e-01  2.470e-01   0.507 0.612020
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 8927.2  on 10126  degrees of freedom
## Residual deviance: 4710.6  on 10095  degrees of freedom
## AIC: 4774.6
##
## Number of Fisher Scoring iterations: 6
```

“จากการพิจารณา P-value ทำให้ตัวแปรสุทธิที่จะถูกคัดเลือกคือ

y = Attrition_flag

x1 = Gender,

x2 = Dependent_count

x3 =Income_Category

x4 = Total_Relationship_Count

x5 = Months_Inactive_12_mon

x6 = Contacts_Count_12_mon

x7 = Total_Revolving_Bal

x8 = Total_Trans_Amt

x9 = Total_Trans_Ct

“ทำ Monte แบ่ง Data หาจุดตัว prob ที่ดีที่สุด และ หา Average Accuracy”

```
monte_acc <- seq(20)
each_threshold <- seq(0.1,0.9,by = 0.1)

tp_each_monte <- seq(20)
tn_each_monte <- seq(20)
fp_each_monte <- seq(20)
fn_each_monte <- seq(20)

pre_each_monte <- seq(20)
rec_each_monte <- seq(20)

f_1_each_monte <- seq(20)

for (i in 1:20){
  for_train <- sample(number, size = (0.8 * number),replace = FALSE)
  for (j in 1:length(each_threshold)){
    fin_model <- glm(Attrition_Flag ~
                       Gender + Dependent_count + Income_Category +
                       Total_Relationship_Count + Months_Inactive_12_mon +
Contacts_Count_12_mon + Total_Revolving_Bal + Total_Trans_Amt +
Total_Trans_Ct,
                     data = bank, subset = for_train, family = 'binomial' )
    fin_prob <- predict(fin_model, newdata = bank[-for_train,], type =
'response')

    fin_pred <- rep("Attrited Customer",length(fin_prob))
```

```
    fin_pred[fin_prob > (j/10)] = "Existing Customer"

    each_threshold[j] = mean(fin_pred == bank[-for_train, ]$Attrition_Flag)
  }
  fin_prob <- predict(fin_model, newdata = bank[-for_train,], type =
'response')
  fin_pred <- rep("Attrited Customer",length(fin_prob))
  fin_pred[fin_prob > (which.max(each_threshold)/10)] = "Existing Customer"

  monte_acc[i] =  mean(fin_pred == bank[-for_train, ]$Attrition_Flag)

  fin_table <- table(fin_pred, bank[-for_train, ]$Attrition_Flag)
  tp_each_monte[i] <- fin_table[2,2]
  tn_each_monte[i] <- fin_table[1,1]
  fp_each_monte[i] <- fin_table[2,1]
  fn_each_monte[i] <- fin_table[1,2]

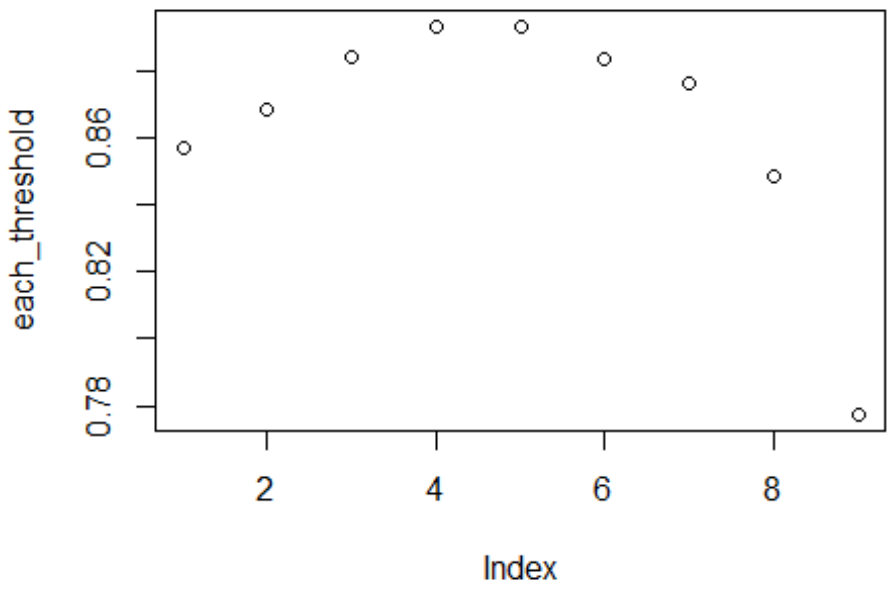
  pre_each_monte[i] <- tp_each_monte[i]/(tp_each_monte[i]+fp_each_monte[i])
  rec_each_monte[i] <- tp_each_monte[i]/(tp_each_monte[i]+fn_each_monte[i])

  f_1_each_monte[i] <- 2*(pre_each_monte[i] *
rec_each_monte[i])/(pre_each_monte[i] + rec_each_monte[i])
}
```

“จากการทำ Monte 20 รอบ ที่ train st = 80% ได้ค่าเป็น”

“Best threshold”

```
plot(each_threshold)
```




```
cut_prob = which.max(each_threshold)/10
cut_prob

## [1] 0.4
```

“Bank Model Accuracy”

```
monte_acc

##  [1] 0.8993090 0.9007897 0.8958539 0.8874630 0.9027641 0.8899309 0.8933860
##  [8] 0.8948667 0.8894373 0.8958539 0.9072063 0.8938796 0.8933860 0.8988154
## [15] 0.9042448 0.8894373 0.8958539 0.9027641 0.8948667 0.8933860
```

“Bank Model True Positive”

```
tp_each_monte

##  [1] 1610 1641 1641 1630 1648 1683 1646 1638 1601 1653 1664 1623 1642 1678
1649
## [16] 1648 1673 1638 1661 1663
```

“Bank Model True Negative”

```
tn_each_monte

##  [1] 212 184 174 168 181 120 164 175 201 162 174 188 168 143 183 154 142
191 152
## [20] 147
```

“Bank Model False Positive”

```
fp_each_monte
```

```
##  [1] 117 147 156 169 135 168 146 147 139 149 129 151 160 172 143 172 180
133 170
## [20] 174
```

“Bank Model False Negative”

```
fn_each_monte
```

```
##  [1] 87 54 55 59 62 55 70 66 85 62 59 64 56 33 51 52 31 64 43 42
```

“Bank Model Precision”

```
pre_each_monte
```

```
##  [1] 0.9322525 0.9177852 0.9131886 0.9060589 0.9242849 0.9092382 0.9185268
##  [8] 0.9176471 0.9201149 0.9173141 0.9280535 0.9148816 0.9112098 0.9070270
## [15] 0.9202009 0.9054945 0.9028602 0.9249012 0.9071546 0.9052803
```

“Bank Model Recall”

```
rec_each_monte
```

```
##  [1] 0.9487331 0.9681416 0.9675708 0.9650681 0.9637427 0.9683544 0.9592075
##  [8] 0.9612676 0.9495848 0.9638484 0.9657574 0.9620628 0.9670200 0.9807130
## [15] 0.9700000 0.9694118 0.9818075 0.9623972 0.9747653 0.9753666
```

“Bank Model F1 Score”

```
f_1_each_monte
```

```
##  [1] 0.9404206 0.9422911 0.9395935 0.9346330 0.9436015 0.9378657 0.9384265
##  [8] 0.9389510 0.9346176 0.9400057 0.9465301 0.9378792 0.9382857 0.9424319
## [15] 0.9444444 0.9363636 0.9406803 0.9432767 0.9397454 0.9390175
```

“Average Bank Model Accuracy”

```
avg_acc <- mean(monte_acc)
avg_acc
```

```
## [1] 0.8961747
```

“Average Bank Model True Positive”

```
avg_tp <- mean(tp_each_monte)
avg_tp
```

```
## [1] 1646.5
```

“Average Bank Model True Negative”

```
avg_tn <- mean(tn_each_monte)
avg_tn
```

```
## [1] 169.15
```

“Average Bank Model False Positive”

```
avg_fp <- mean(fp_each_monte)
avg_fp
```

```
## [1] 152.85
```

“Average Bank Model False Negative”

```
avg_fn <- mean(fn_each_monte)
avg_fn
```

```
## [1] 57.5
```

“Average Bank Model Precision”

```
avg_precision <- mean(pre_each_monte)
avg_precision
```

```
## [1] 0.9151737
```

“Average Bank Model Recall”

```
avg_recall <- mean(rec_each_monte)
avg_recall
```

```
## [1] 0.966241
```

“Average Bank Model F1 Score”

```
avg_f_1_score <- mean(f_1_each_monte)
avg_f_1_score
```

```
## [1] 0.9399531
```

“Average Bank Model Table”

```
avg_fin_table <- matrix(seq(4),ncol = 2)
colnames(avg_fin_table) <- c('Attrited Customer',  'Existing Customer')
rownames(avg_fin_table) <- c('Predict Attrited Customer',  'Predict Existing
Customer')
avg_fin_table[1,1] <- avg_tn
avg_fin_table[2,2] <- avg_tp
avg_fin_table[1,2] <- avg_fn
avg_fin_table[2,1] <- avg_fp

avg_fin_table
```

```
##                            Attrited Customer Existing Customer
## Predict Attrited Customer            169.15              57.5
## Predict Existing Customer            152.85            1646.5
```

“Bank Model ROC Curve”

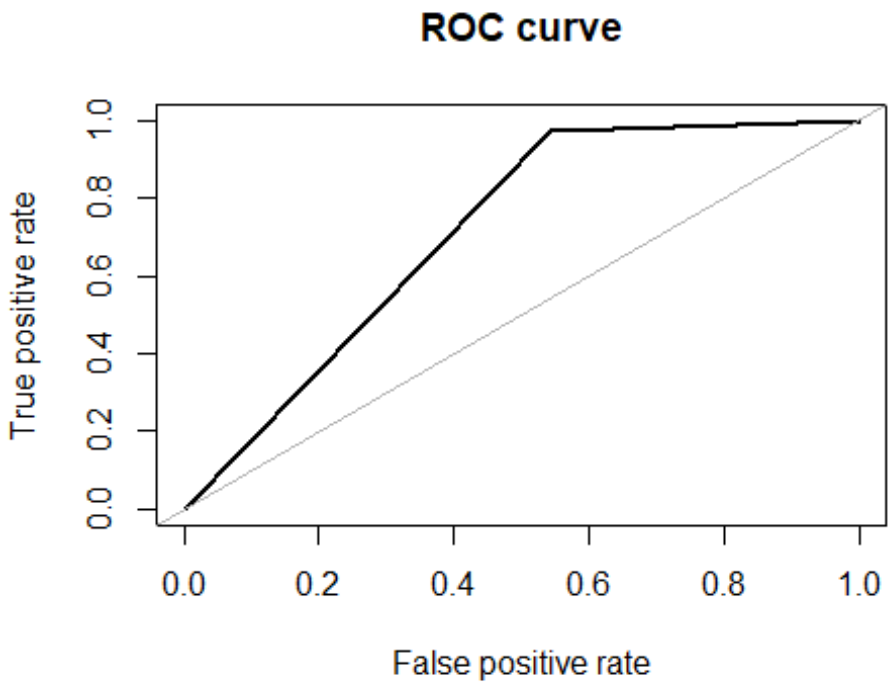
```
library(ROSE)
```

```
## Loaded ROSE 0.0-4
```

```
table(fin_pred, bank$Attrition_Flag[-for_train])
```

```
##
## fin_pred            Attrited Customer Existing Customer
##   Attrited Customer               147                42
##   Existing Customer               174              1663
```

```
roc.curve(bank$Attrition_Flag[-for_train], fin_pred)
```




```
## Area under the curve (AUC): 0.717
```

ข้อ 2

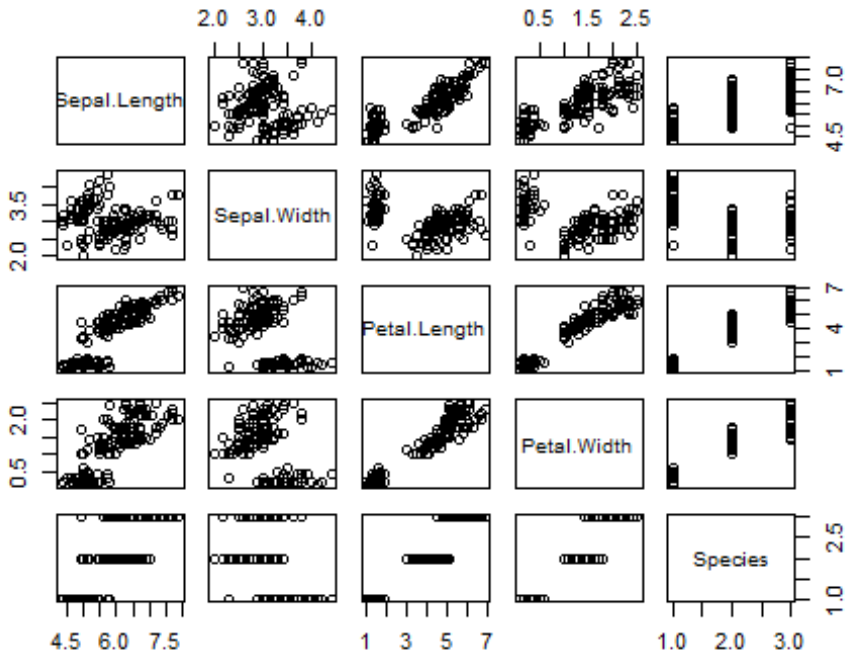
Data : iris

Class Variable : Species

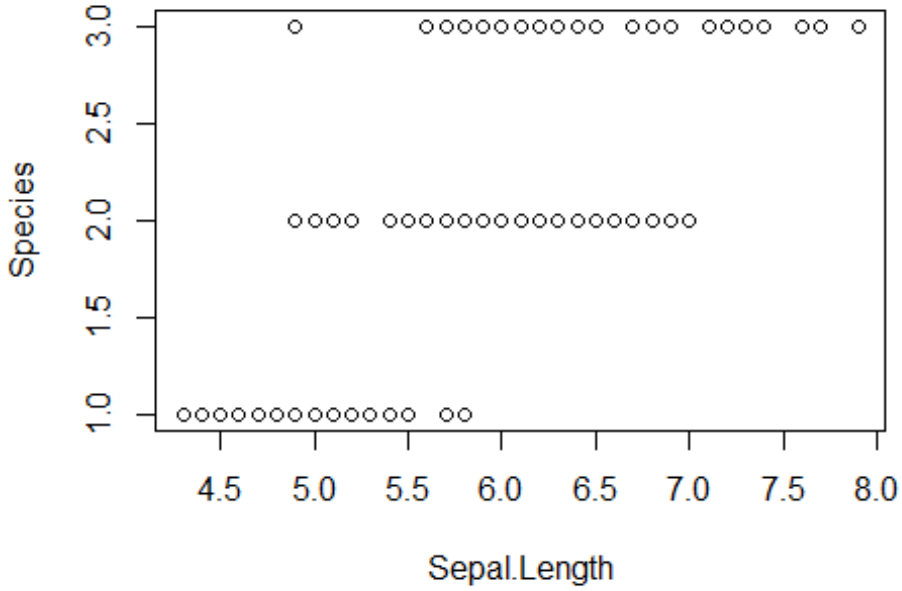
```
table(iris$Species)

##
##     setosa versicolor  virginica
##         50         50         50

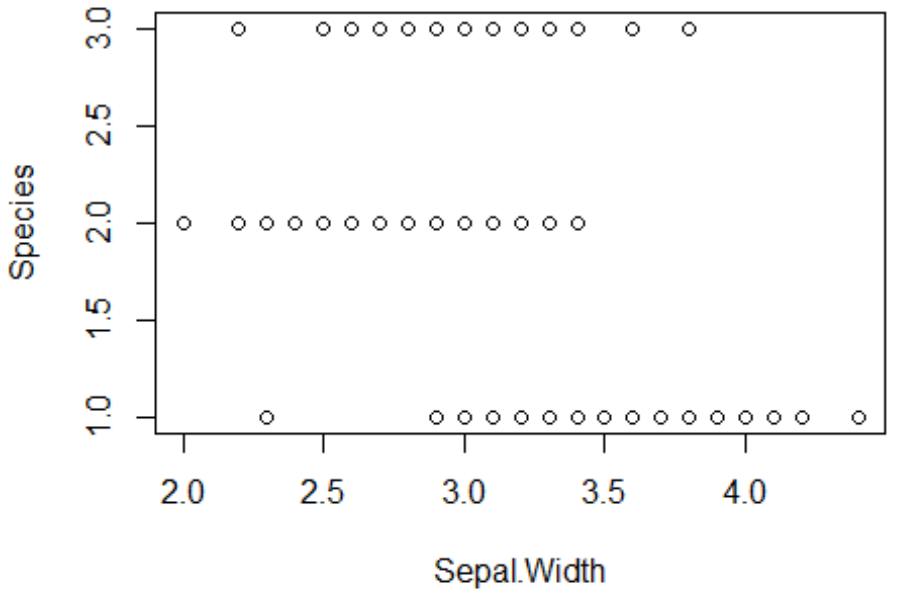
pairs(iris)
```



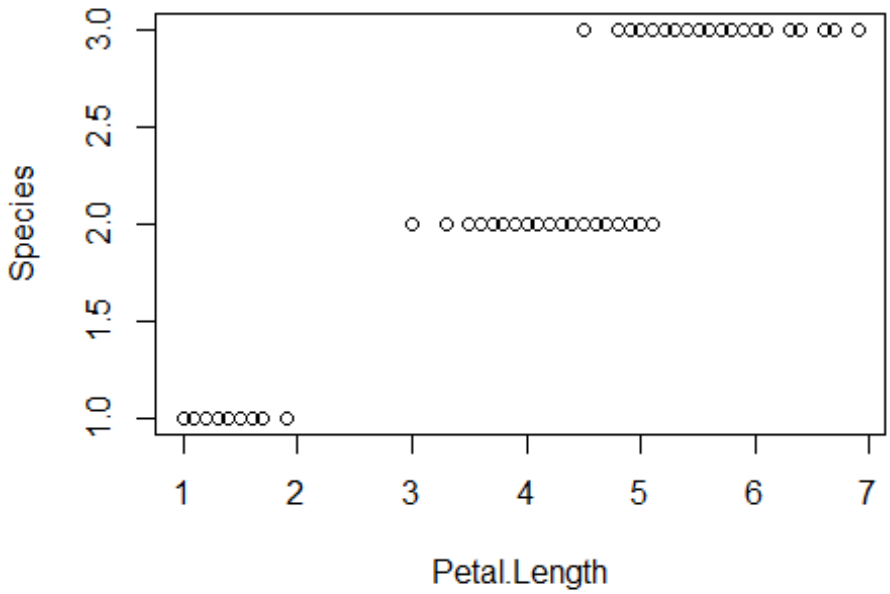

```
plot(iris$Sepal.Length ,iris$Species, xlab = "Sepal.Length", ylab =
"Species")
```



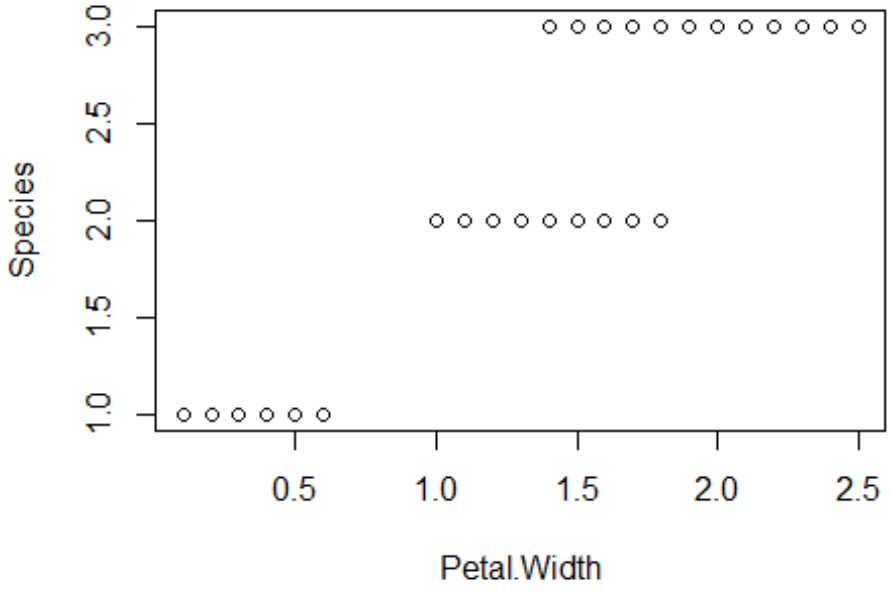

```r
plot(iris$Sepal.Width ,iris$Species, xlab = "Sepal.Width", ylab = "Species")
```



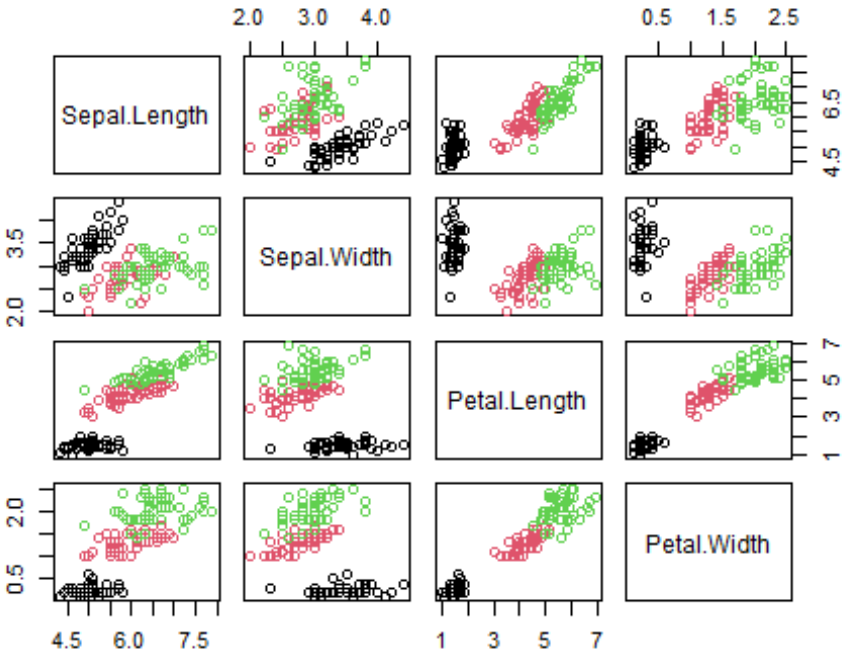
```
plot(iris$Petal.Length ,iris$Species, xlab = "Petal.Length", ylab =
"Species")
```




```
plot(iris$Petal.Width ,iris$Species, xlab = "Petal.Width", ylab = "Species")
```




```
pairs(iris[1:4], col = iris$Species)
```



ข้อ 2.1

“จากการที่ ตัวแปร Class ที่เป็น Norminal Scale (ชนิด Species)

จึงใช้ Multinom ในการสร้างตัวแบบ”

โดยที่

y = Species

x1 = Sepal.Length

x2 = Sepal.Width

x3 = Petal.Length

x4 = Petal.Width

```
full_iris
```

```
## Call:
## multinom(formula = Species ~ ., data = iris, HESS = TRUE)
##
## Coefficients:
##            (Intercept) Sepal.Length Sepal.Width Petal.Length Petal.Width
## versicolor    18.69037    -5.458424   -8.707401     14.24477   -3.097684
## virginica    -23.83628    -7.923634  -15.370769     23.65978   15.135301
##
## Std. Errors:
##            (Intercept) Sepal.Length Sepal.Width Petal.Length Petal.Width
## versicolor    34.97116     89.89215    157.0415     60.19170    45.48852
## virginica     35.76649     89.91153    157.1196     60.46753    45.93406
##
## Residual Deviance: 11.89973
## AIC: 31.89973
```

“Class predict probability”

```
full_iris_prob
```

```
##           setosa   versicolor    virginica
## 1   1.000000e+00 1.526406e-09 2.716417e-36
## 2   9.999996e-01 3.536476e-07 2.883729e-32
## 3   1.000000e+00 4.443506e-08 6.103424e-34
## 4   9.999968e-01 3.163905e-06 7.117010e-31
## 5   1.000000e+00 1.102983e-09 1.289946e-36
## 6   1.000000e+00 3.521573e-10 1.344907e-35
## 7   1.000000e+00 4.098064e-08 3.016154e-33
## 8   1.000000e+00 2.615330e-08 2.972971e-34
## 9   9.999871e-01 1.294210e-05 7.048364e-30
## 10  9.999992e-01 8.386603e-07 1.454198e-32
```

```
## 11  1.000000e+00 2.161864e-10 1.241888e-37
## 12  9.999997e-01 3.238036e-07 1.545112e-32
## 13  9.999992e-01 8.320656e-07 1.402024e-32
## 14  9.999998e-01 1.776283e-07 6.091969e-34
## 15  1.000000e+00 2.490019e-14 4.289244e-44
## 16  1.000000e+00 5.099113e-14 5.053040e-42
## 17  1.000000e+00 1.180774e-12 1.043681e-39
## 18  1.000000e+00 1.119797e-09 1.233997e-35
## 19  1.000000e+00 2.229749e-10 1.278090e-36
## 20  1.000000e+00 3.414358e-10 1.306813e-36
## 21  9.999999e-01 5.088458e-08 1.418328e-33
## 22  1.000000e+00 5.983234e-10 2.761055e-35
## 23  1.000000e+00 3.282647e-11 2.381898e-39
## 24  9.999998e-01 2.467861e-07 6.662407e-30
## 25  9.999768e-01 2.323802e-05 1.868716e-29
## 26  9.999965e-01 3.538327e-06 1.482164e-30
## 27  9.999999e-01 5.849351e-08 6.536682e-32
## 28  1.000000e+00 3.674991e-09 1.310414e-35
## 29  1.000000e+00 2.112377e-09 5.720335e-36
## 30  9.999968e-01 3.188981e-06 7.381858e-31
## 31  9.999956e-01 4.413191e-06 1.554498e-30
## 32  1.000000e+00 1.585769e-09 2.578398e-34
## 33  1.000000e+00 2.696754e-11 2.849881e-40
## 34  1.000000e+00 3.875622e-13 2.425003e-42
## 35  9.999994e-01 6.152555e-07 6.606045e-32
## 36  1.000000e+00 2.079286e-09 5.317228e-36
## 37  1.000000e+00 4.138112e-11 1.071492e-38
## 38  1.000000e+00 2.595111e-09 6.271520e-37
## 39  9.999987e-01 1.303796e-06 1.422388e-31
## 40  1.000000e+00 1.515201e-08 1.346082e-34
## 41  1.000000e+00 4.651074e-10 2.558009e-36
## 42  9.997542e-01 2.458213e-04 1.376952e-26
## 43  9.999998e-01 2.285045e-07 6.575528e-33
## 44  1.000000e+00 1.317919e-08 2.900340e-31
## 45  9.999999e-01 7.470478e-08 7.649899e-32
## 46  9.999996e-01 4.478126e-07 2.893285e-31
## 47  1.000000e+00 1.934115e-09 3.064974e-36
## 48  9.999997e-01 3.187312e-07 1.436229e-32
## 49  1.000000e+00 3.731511e-10 2.742847e-37
## 50  1.000000e+00 1.503286e-08 1.297787e-34
## 51  2.427101e-07 9.999877e-01 1.201699e-05
## 52  2.160475e-07 9.999501e-01 4.968516e-05
## 53  4.640834e-09 9.987828e-01 1.217158e-03
## 54  4.185792e-10 9.999567e-01 4.326447e-05
## 55  2.752538e-09 9.985711e-01 1.428890e-03
## 56  7.824187e-11 9.998954e-01 1.045901e-04
## 57  2.356899e-08 9.986727e-01 1.327314e-03
## 58  3.195371e-07 9.999997e-01 5.641233e-10
## 59  6.116463e-09 9.999850e-01 1.497847e-05
## 60  1.501151e-08 9.999848e-01 1.523161e-05
```

```
## 61  9.809848e-10 1.000000e+00 4.165185e-08
## 62  1.773719e-07 9.999615e-01 3.834000e-05
## 63  1.060055e-09 9.999999e-01 1.034374e-07
## 64  1.308456e-10 9.991850e-01 8.150241e-04
## 65  4.002682e-05 9.999600e-01 1.436141e-08
## 66  1.418052e-06 9.999957e-01 2.908759e-06
## 67  4.799737e-10 9.986481e-01 1.351871e-03
## 68  6.658268e-09 1.000000e+00 1.551529e-08
## 69  1.127345e-11 9.401019e-01 5.989806e-02
## 70  9.220385e-09 9.999999e-01 9.072544e-08
## 71  2.958914e-10 5.945365e-01 4.054635e-01
## 72  8.608392e-07 9.999988e-01 3.522422e-07
## 73  7.324234e-13 7.743208e-01 2.256792e-01
## 74  2.950369e-11 9.999586e-01 4.141866e-05
## 75  1.473401e-07 9.999984e-01 1.455234e-06
## 76  3.439354e-07 9.999924e-01 7.246952e-06
## 77  6.017178e-10 9.992755e-01 7.245125e-04
## 78  2.112470e-10 7.236305e-01 2.763695e-01
## 79  1.784210e-09 9.990177e-01 9.822717e-04
## 80  8.317614e-06 9.999917e-01 1.361048e-10
## 81  9.293464e-09 9.999999e-01 8.816365e-08
## 82  2.833280e-08 1.000000e+00 5.553317e-09
## 83  2.136523e-07 9.999997e-01 9.050639e-08
## 84  1.096390e-14 1.323524e-01 8.676476e-01
## 85  1.609647e-10 9.977885e-01 2.211499e-03
## 86  1.892766e-07 9.997823e-01 2.175106e-04
## 87  2.692561e-08 9.996965e-01 3.034535e-04
## 88  1.105514e-10 9.997399e-01 2.600700e-04
## 89  7.714596e-08 9.999991e-01 8.170920e-07
## 90  2.388398e-09 9.999886e-01 1.141228e-05
## 91  1.403301e-11 9.999591e-01 4.089587e-05
## 92  1.299698e-09 9.998366e-01 1.633724e-04
## 93  2.152323e-08 9.999995e-01 4.518083e-07
## 94  2.308979e-07 9.999998e-01 8.584159e-10
## 95  1.362045e-09 9.999845e-01 1.546367e-05
## 96  2.350697e-08 9.999997e-01 2.643923e-07
## 97  1.341431e-08 9.999968e-01 3.187736e-06
## 98  4.945474e-08 9.999976e-01 2.382636e-06
## 99  2.224095e-04 9.997776e-01 6.500522e-11
## 100 2.333746e-08 9.999976e-01 2.420920e-06
## 101 9.453717e-25 2.718072e-10 1.000000e+00
## 102 2.762230e-17 3.922358e-04 9.996078e-01
## 103 2.413930e-20 9.974371e-07 9.999990e-01
## 104 1.039086e-18 2.851578e-04 9.997148e-01
## 105 4.877802e-22 9.409138e-08 9.999999e-01
## 106 8.139586e-26 4.698713e-09 1.000000e+00
## 107 2.747116e-14 1.091926e-01 8.908074e-01
## 108 1.841814e-22 4.609074e-06 9.999954e-01
## 109 4.655966e-22 8.093448e-06 9.999919e-01
## 110 1.116285e-20 7.196079e-09 1.000000e+00
```

```
## 111 3.360175e-12 9.861345e-03 9.901387e-01
## 112 2.824675e-17 2.619406e-04 9.997381e-01
## 113 2.887245e-17 2.057044e-05 9.999794e-01
## 114 1.356507e-18 3.348943e-05 9.999665e-01
## 115 6.643324e-20 8.391928e-08 9.999999e-01
## 116 1.443873e-16 4.987152e-06 9.999950e-01
## 117 2.506556e-16 2.325939e-03 9.976741e-01
## 118 8.132508e-22 7.823403e-08 9.999999e-01
## 119 1.539275e-32 6.473411e-13 1.000000e+00
## 120 2.586465e-16 7.964338e-02 9.203566e-01
## 121 5.888460e-19 3.959256e-07 9.999996e-01
## 122 6.580602e-16 4.950994e-04 9.995049e-01
## 123 3.543398e-27 3.830263e-09 1.000000e+00
## 124 7.099730e-13 5.193896e-02 9.480610e-01
## 125 1.158605e-17 1.805360e-05 9.999819e-01
## 126 1.014284e-17 4.479026e-04 9.995521e-01
## 127 1.384328e-11 1.760948e-01 8.239052e-01
## 128 1.238609e-11 1.980731e-01 8.019269e-01
## 129 5.264982e-21 7.894776e-07 9.999992e-01
## 130 1.067125e-15 2.892881e-02 9.710712e-01
## 131 2.185577e-21 3.215285e-06 9.999968e-01
## 132 9.900467e-17 8.276525e-05 9.999172e-01
## 133 1.158989e-21 1.274946e-07 9.999999e-01
## 134 5.926801e-13 7.939466e-01 2.060534e-01
## 135 8.716903e-19 3.353546e-02 9.664645e-01
## 136 1.196029e-21 1.736953e-08 1.000000e+00
## 137 2.573884e-19 1.415958e-07 9.999999e-01
## 138 5.272004e-16 3.535048e-03 9.964650e-01
## 139 4.984248e-11 3.310585e-01 6.689415e-01
## 140 3.159583e-15 1.313812e-04 9.998686e-01
## 141 6.087418e-20 5.142118e-08 9.999999e-01
## 142 1.851909e-13 5.774763e-05 9.999423e-01
## 143 2.762230e-17 3.922358e-04 9.996078e-01
## 144 2.348662e-21 4.707320e-08 1.000000e+00
## 145 2.720648e-20 1.227942e-08 1.000000e+00
## 146 7.661759e-16 7.065708e-06 9.999929e-01
## 147 7.146172e-16 9.093936e-04 9.990906e-01
## 148 1.470964e-14 1.023609e-03 9.989764e-01
## 149 6.009635e-17 4.504137e-06 9.999955e-01
## 150 2.726745e-14 2.243538e-02 9.775646e-01
```

“Class predict”

```
full_iris_pred
```

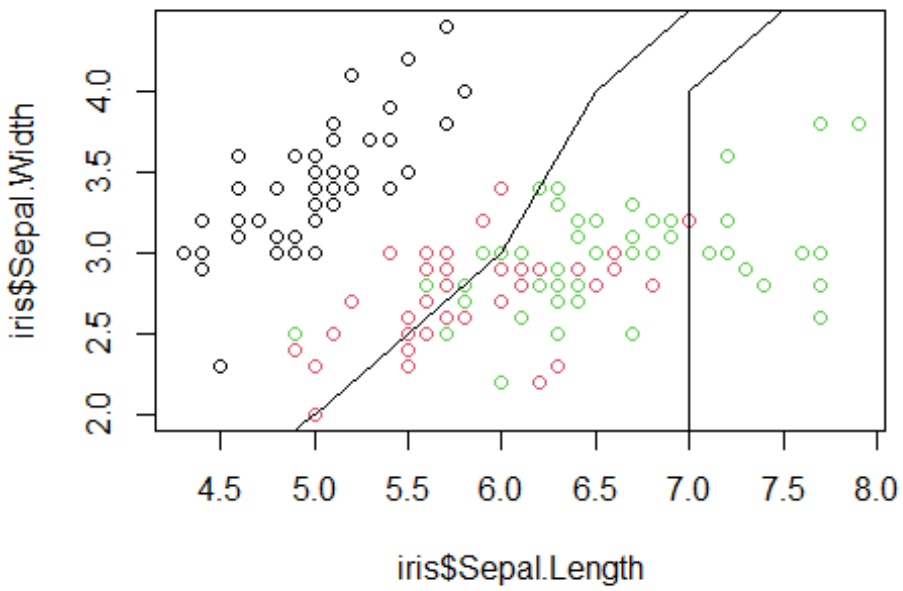
```
##   [1] setosa     setosa     setosa     setosa     setosa     setosa
##   [7] setosa     setosa     setosa     setosa     setosa     setosa
##  [13] setosa     setosa     setosa     setosa     setosa     setosa
##  [19] setosa     setosa     setosa     setosa     setosa     setosa
##  [25] setosa     setosa     setosa     setosa     setosa     setosa
##  [31] setosa     setosa     setosa     setosa     setosa     setosa
```

```
##  [37] setosa     setosa     setosa     setosa     setosa     setosa
##  [43] setosa     setosa     setosa     setosa     setosa     setosa
##  [49] setosa     setosa     versicolor versicolor versicolor versicolor
##  [55] versicolor versicolor versicolor versicolor versicolor versicolor
##  [61] versicolor versicolor versicolor versicolor versicolor versicolor
##  [67] versicolor versicolor versicolor versicolor versicolor versicolor
##  [73] versicolor versicolor versicolor versicolor versicolor versicolor
##  [79] versicolor versicolor versicolor versicolor versicolor virginica
##  [85] versicolor versicolor versicolor versicolor versicolor versicolor
##  [91] versicolor versicolor versicolor versicolor versicolor versicolor
##  [97] versicolor versicolor versicolor versicolor virginica  virginica
## [103] virginica  virginica  virginica  virginica  virginica  virginica
## [109] virginica  virginica  virginica  virginica  virginica  virginica
## [115] virginica  virginica  virginica  virginica  virginica  virginica
## [121] virginica  virginica  virginica  virginica  virginica  virginica
## [127] virginica  virginica  virginica  virginica  virginica  virginica
## [133] virginica  versicolor virginica  virginica  virginica  virginica
## [139] virginica  virginica  virginica  virginica  virginica  virginica
## [145] virginica  virginica  virginica  virginica  virginica  virginica
## Levels: setosa versicolor virginica
```

Discriminant Line

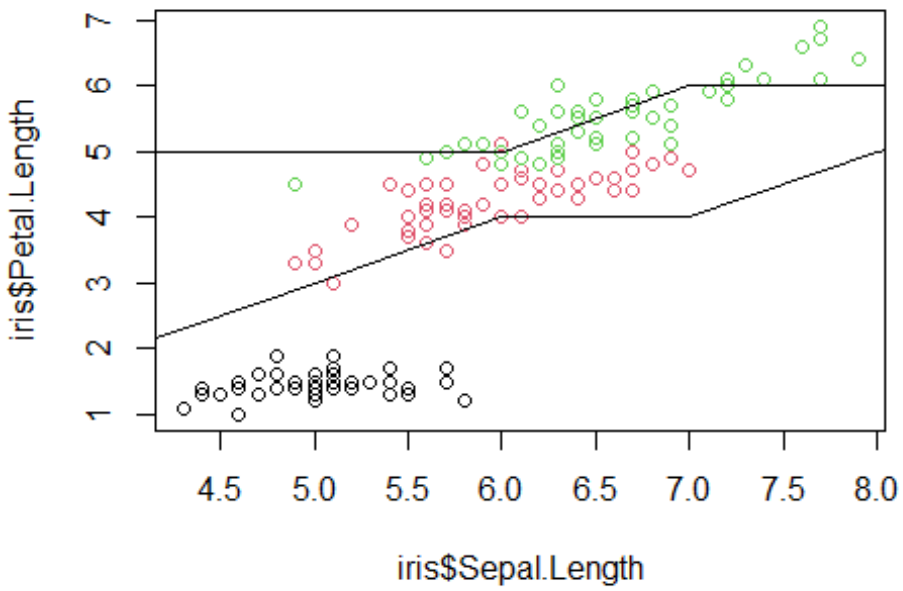
Sepal.Length - Sepal.Width

```
plot(iris$Sepal.Length,iris$Sepal.Width, col = iris$Species)
contour(x  = sep_length, y = sep_width, z = matrix(prd_1_2, nrow = np , ncol
= np), levels = c(1, 2,3), add = TRUE, drawlabels = FALSE)
```



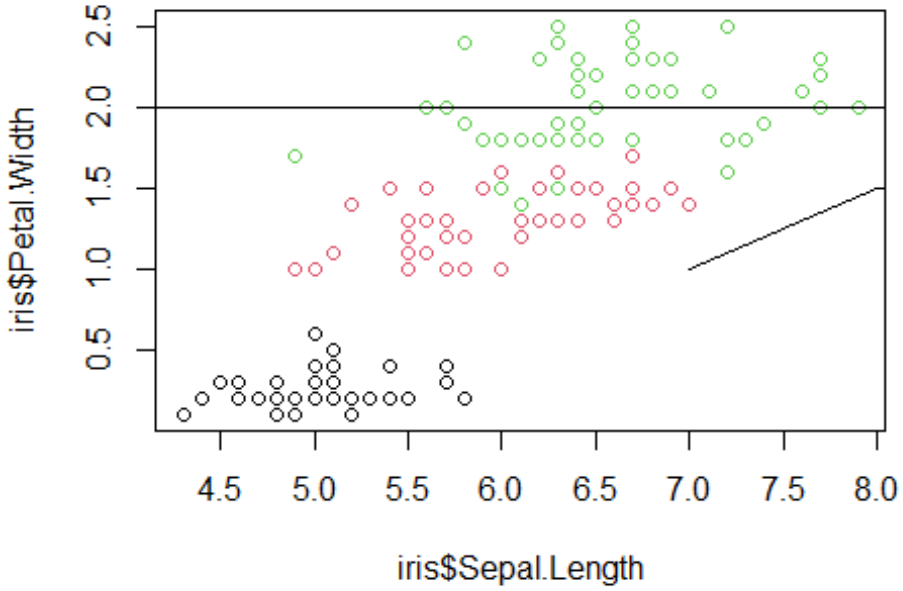
Sepal.Length - Petal.Length

```
plot(iris$Sepal.Length,iris$Petal.Length, col = iris$Species)
contour(x  = sep_length, y = pet_length, z = matrix(prd_1_3, nrow = np , ncol
= np), levels = c(1, 2,3), add = TRUE, drawlabels = FALSE)
```



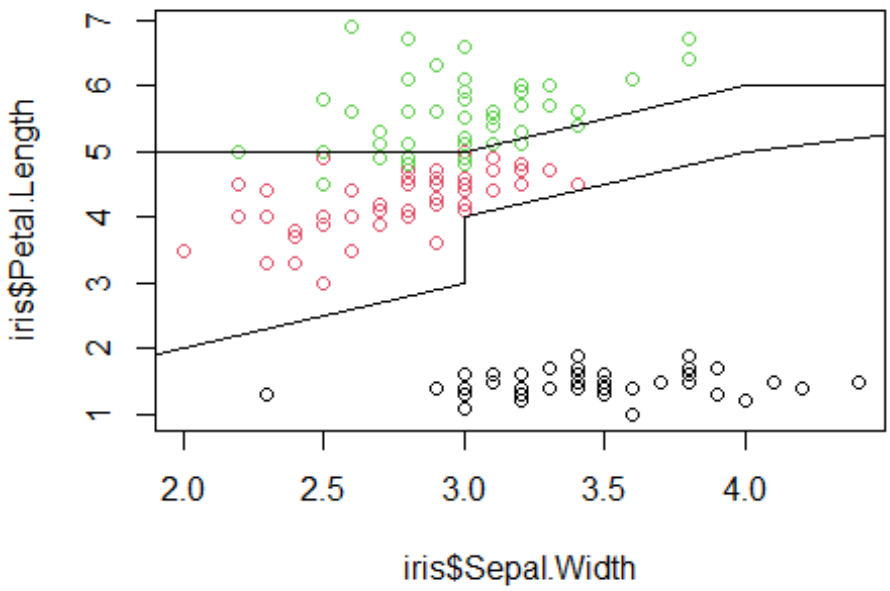
Sepal.Length - Petal.Width

```
plot(iris$Sepal.Length,iris$Petal.Width, col = iris$Species)
contour(x  = sep_length, y = pet_width, z = matrix(prd_1_4, nrow = np , ncol
= np), levels = c(1, 2,3), add = TRUE, drawlabels = FALSE)
```



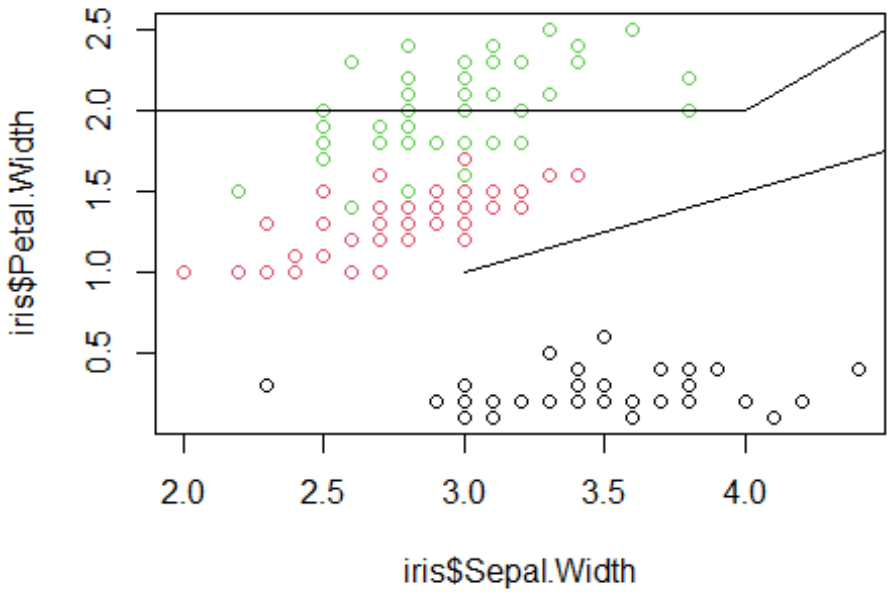
Sepal.Width - Petal.Length

```
plot(iris$Sepal.Width,iris$Petal.Length, col = iris$Species)
contour(x  = sep_width, y = pet_length, z = matrix(prd_2_3, nrow = np , ncol
= np), levels = c(1, 2,3), add = TRUE, drawlabels = FALSE)
```



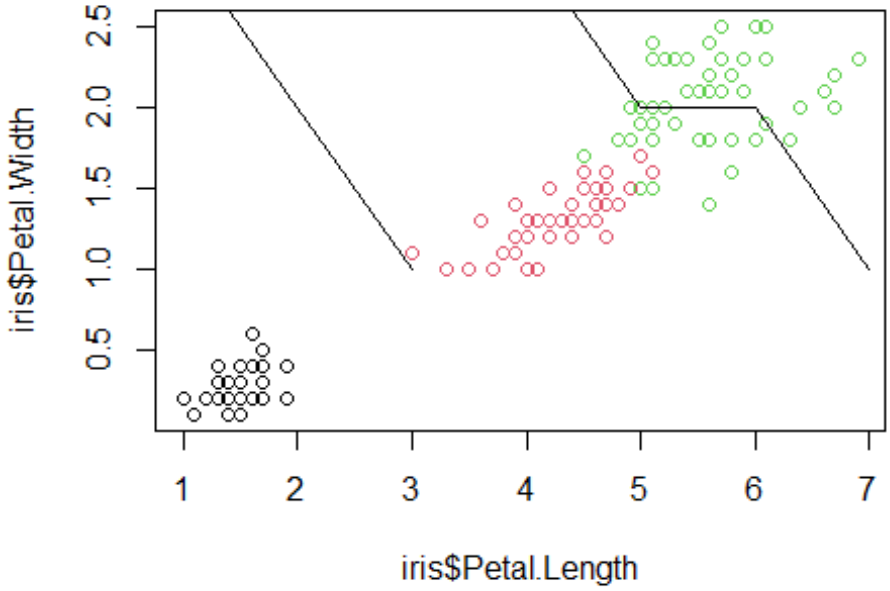
Sepal.Width - Petal.Width

```
plot(iris$Sepal.Width,iris$Petal.Width, col = iris$Species)
contour(x  = sep_width, y = pet_width, z = matrix(prd_2_4, nrow = np , ncol = np), levels = c(1, 2,3), add = TRUE, drawlabels = FALSE)
```



Petal.Length - Petal.Width

```
plot(iris$Petal.Length,iris$Petal.Width, col = iris$Species)
contour(x  = pet_length, y = pet_width, z = matrix(prd_3_4, nrow = np , ncol = np), levels = c(1, 2,3), add = TRUE, drawlabels = FALSE)
```



“Predict Table”

```
full_iris_table
```

```
##
##              setosa versicolor virginica
##   setosa         50          0         0
##   versicolor      0         49         1
##   virginica       0          1        49
```

“Accuracy”

```
full_iris_acc
```

```
## [1] 0.9866667
```

ข้อ 2.2

“Monte Carlo 20 รอบ ด้วย train_data = 80%”

“iris Multinom Accuracy”

```
model.acc
```

```
##  [1] 0.9666667 1.0000000 1.0000000 0.9666667 1.0000000 1.0000000 0.9000000
##  [8] 0.9333333 0.9000000 1.0000000 1.0000000 0.9666667 0.9666667 0.9666667
## [15] 1.0000000 0.9333333 0.9000000 0.9000000 1.0000000 0.9333333
```

“Average iris Multinom Accuracy”

```
mean(model.acc)
```

```
## [1] 0.9616667
```

“iris Multinom Precision”

Class 1 (Setosa)

```
model.pre.c1
```

```
##  [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

Class 2 (Versicolor)

```
model.pre.c2
```

```
##  [1] 1.0000000 1.0000000 1.0000000 0.8888889 1.0000000 1.0000000 0.8750000
##  [8] 1.0000000 0.7500000 1.0000000 1.0000000 1.0000000 0.8888889 0.9090909
## [15] 1.0000000 1.0000000 0.8333333 0.7500000 1.0000000 0.8666667
```

Class 3 (Virginica)

```
model.pre.c3
```

```
##  [1] 0.8750000 1.0000000 1.0000000 1.0000000 1.0000000 1.0000000 0.8750000
##  [8] 0.7777778 1.0000000 1.0000000 1.0000000 0.8571429 1.0000000 1.0000000
## [15] 1.0000000 0.8333333 0.8571429 1.0000000 1.0000000 1.0000000
```

“Average iris Multinom Precision”

Average Class 1 (Setosa)

```
mean(model.pre.c1)
```

```
## [1] 1
```

Average Class 2 (Versicolor)

```
mean(model.pre.c2)
```

```
## [1] 0.9380934
```

Average Class 3 (Virginica)

```
mean(model.pre.c3)
```

```
## [1] 0.9537698
```

“iris Multinom Recall”

Class 1 (Setosa)

```
model.rec.c1
```

```
##  [1] 1.0000000 1.0000000 1.0000000 1.0000000 1.0000000 1.0000000 0.8571429
##  [8] 1.0000000 1.0000000 1.0000000 1.0000000 1.0000000 1.0000000 1.0000000
## [15] 1.0000000 1.0000000 1.0000000 1.0000000 1.0000000 1.0000000
```

Class 2 (Versicolor)

```
model.rec.c2
```

```
##  [1] 0.9230769 1.0000000 1.0000000 1.0000000 1.0000000 1.0000000 0.7777778
##  [8] 0.8181818 1.0000000 1.0000000 1.0000000 0.9166667 1.0000000 1.0000000
## [15] 1.0000000 0.8333333 0.9090909 1.0000000 1.0000000 1.0000000
```

Class 3 (Virginica)

```
model.rec.c3
```

```
##  [1] 1.0000000 1.0000000 1.0000000 0.8888889 1.0000000 1.0000000 1.0000000
##  [8] 1.0000000 0.7000000 1.0000000 1.0000000 1.0000000 0.9090909 0.9000000
## [15] 1.0000000 1.0000000 0.7500000 0.7692308 1.0000000 0.8333333
```

“Average iris Multinom Recall”

Average Class 1 (Setosa)

```
mean(model.rec.c1)
```

```
## [1] 0.9928571
```

Average Class 2 (Versicolor)

```
mean(model.rec.c2)
```

```
## [1] 0.9589064
```

Average Class 3 (Virginica)

```
mean(model.rec.c3)
```

```
## [1] 0.9375272
```

ข้อ 2.3

“ทำ Monte 20 รอบ แล้ว หา Average Accuracy ของแต่ละระดับค่า K”

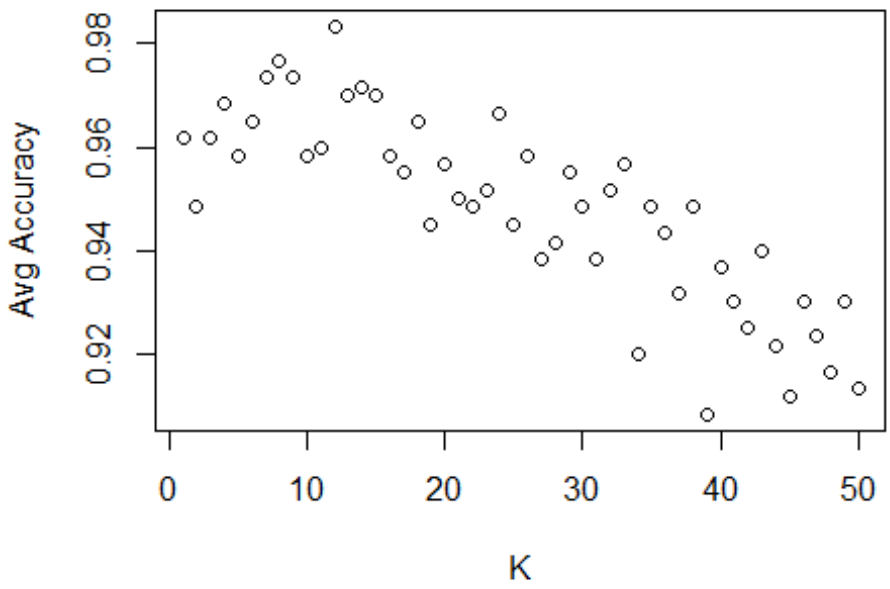
“Average Accuracy แต่ละระดับค่า K”

```
iris_knn_acc
```

```
##        K Avg Accuracy
##  [1,]  1    0.9616667
##  [2,]  2    0.9483333
##  [3,]  3    0.9616667
##  [4,]  4    0.9683333
##  [5,]  5    0.9583333
##  [6,]  6    0.9650000
##  [7,]  7    0.9733333
##  [8,]  8    0.9766667
##  [9,]  9    0.9733333
## [10,] 10    0.9583333
```

```
## [11,] 11    0.9600000
## [12,] 12    0.9833333
## [13,] 13    0.9700000
## [14,] 14    0.9716667
## [15,] 15    0.9700000
## [16,] 16    0.9583333
## [17,] 17    0.9550000
## [18,] 18    0.9650000
## [19,] 19    0.9450000
## [20,] 20    0.9566667
## [21,] 21    0.9500000
## [22,] 22    0.9483333
## [23,] 23    0.9516667
## [24,] 24    0.9666667
## [25,] 25    0.9450000
## [26,] 26    0.9583333
## [27,] 27    0.9383333
## [28,] 28    0.9416667
## [29,] 29    0.9550000
## [30,] 30    0.9483333
## [31,] 31    0.9383333
## [32,] 32    0.9516667
## [33,] 33    0.9566667
## [34,] 34    0.9200000
## [35,] 35    0.9483333
## [36,] 36    0.9433333
## [37,] 37    0.9316667
## [38,] 38    0.9483333
## [39,] 39    0.9083333
## [40,] 40    0.9366667
## [41,] 41    0.9300000
## [42,] 42    0.9250000
## [43,] 43    0.9400000
## [44,] 44    0.9216667
## [45,] 45    0.9116667
## [46,] 46    0.9300000
## [47,] 47    0.9233333
## [48,] 48    0.9166667
## [49,] 49    0.9300000
## [50,] 50    0.9133333
```

```
plot(iris_knn_acc)
```



“ค่า K ที่ Average Accuracy มากที่สุด”

```
best_k <- which.max(iris_knn_acc[ ,2])
best_k
```

```
## [1] 12
```

“ทำ KNN คู่ กับ Monte train data 80%

ด้วยค่า K ที่ทำให้ได้ Average Accuracy มากที่สุด”

“KNN iris Accuracy”

```
knn.model.acc
```

```
##  [1] 0.9666667 0.9000000 0.9333333 0.9666667 0.9333333 1.0000000 0.9333333
##  [8] 1.0000000 0.9666667 1.0000000 0.9333333 0.9000000 0.9666667 0.9666667
## [15] 0.9333333 1.0000000 0.9666667 0.9333333 0.9333333 0.9333333
```

“Average KNN iris Accuracy”

```
mean(knn.model.acc)
```

```
## [1] 0.9533333
```

“KNN iris Precision”

Class 1 (Setosa)

```
knn.model.pre.c1
```

```
##  [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

Class 2 (Versicolor)

```
knn.model.pre.c2
```

```
##  [1] 1.0000000 0.9000000 0.8000000 0.8888889 1.0000000 1.0000000 1.0000000
##  [8] 1.0000000 1.0000000 1.0000000 0.9090909 0.8333333 0.8888889 1.0000000
## [15] 0.8888889 1.0000000 0.8333333 0.8571429 1.0000000 0.8333333
```

Class 3 (Virginica)

```
knn.model.pre.c3
```

```
##  [1] 0.8333333 0.8000000 1.0000000 1.0000000 0.8333333 1.0000000 0.8000000
##  [8] 1.0000000 0.9090909 1.0000000 0.8750000 0.9090909 1.0000000 0.9166667
## [15] 0.8571429 1.0000000 1.0000000 0.9230769 0.8181818 1.0000000
```

“Average KNN iris Precision”

Average Class 1 (Setosa)

```
mean(knn.model.pre.c1)
```

```
## [1] 1
```

Average KNN Class 2 (Versicolor)

```
mean(knn.model.pre.c2)
```

```
## [1] 0.931645
```

Average KNN Class 3 (Virginica)

```
mean(knn.model.pre.c3)
```

```
## [1] 0.9237458
```

“KNN iris Recall”

Class 1 (Setosa)

```
knn.model.rec.c1
```

```
##  [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

Class 2 (Versicolor)

```
knn.model.rec.c2
```

```
##  [1] 0.9375000 0.8181818 1.0000000 1.0000000 0.8181818 1.0000000 0.8181818
##  [8] 1.0000000 0.8888889 1.0000000 0.9090909 0.9090909 1.0000000 0.9090909
## [15] 0.8888889 1.0000000 1.0000000 0.8571429 0.8000000 1.0000000
```

Class 3 (Virginica)

```
knn.model.rec.c3
```

```
##  [1] 1.0000000 0.8888889 0.8461538 0.8750000 1.0000000 1.0000000 1.0000000
##  [8] 1.0000000 1.0000000 1.0000000 0.8750000 0.8333333 0.9230769 1.0000000
## [15] 0.8571429 1.0000000 0.9166667 0.9230769 1.0000000 0.7500000
```

“Average KNN iris Recall”

Average Class 1 (Setosa)

```
mean(knn.model.rec.c1)
```

```
## [1] 1
```

Average Class 2 (Versicolor)

```
mean(knn.model.rec.c2)
```

```
## [1] 0.9277119
```

Average Class 3 (Virginica)

```
mean(knn.model.rec.c3)
```

```
## [1] 0.934417
```

Compare KNN กับ Multinom

ตาราง Average

```
avg_compare_table <- matrix(seq(14), ncol = 2)
colnames(avg_compare_table) <- c("Multinom", "KNN")
rownames(avg_compare_table) <- c("Accuracy", "Precision Class 1 (Setosa)",
"Precision Class 2 (Versicolor)", "Precision Class 3 (Virginica)", "Recall
Class 1 (Setosa)", "Recall Class 2 (Versicolor)", "Recall Class 3
(Virginica)" )

avg_compare_table[, 1] =
c(mean(model.acc),mean(model.pre.c1),mean(model.pre.c2),mean(model.pre.c3),me
an(model.rec.c1),mean(model.rec.c2),mean(model.rec.c3))

avg_compare_table[, 2] =
c(mean(knn.model.acc),mean(knn.model.pre.c1),mean(knn.model.pre.c2),mean(knn.
model.pre.c3),mean(knn.model.rec.c1),mean(knn.model.rec.c2),mean(knn.model.re
c.c3))

avg_compare_table
```

```
##                                Multinom       KNN
## Accuracy                      0.9616667 0.9533333
## Precision Class 1 (Setosa)    1.0000000 1.0000000
## Precision Class 2 (Versicolor) 0.9380934 0.9316450
## Precision Class 3 (Virginica)  0.9537698 0.9237458
```

```
## Recall Class 1 (Setosa)        0.9928571 1.0000000
## Recall Class 2 (Versicolor)    0.9589064 0.9277119
## Recall Class 3 (Virginica)     0.9375272 0.9344170
```